

PHẦN 8.3

HASHSET<T> - TẬP HỢP KHÔNG TRÙNG LẶP

Mục tiêu: Hiểu cách lưu dữ liệu duy nhất, thêm/xóa/kiểm tra phần tử, đóng gói HashSet và chọn đúng collection cho từng bài toán.

Nội dung chính	Bài tập cuối bài
Add, Contains, Remove, Count, Clear, StringComparer, các phép toán tập hợp và Encapsulation.	StudentCourseRegistration - quản lý mã môn học đã đăng ký, không cho trùng.

Tài liệu ôn tập cá nhân - Không kèm đáp án bài tập

Mục lục

1. HashSet là gì?
2. Cú pháp và quy tắc không trùng
3. Add() trả về bool
4. Contains(), Remove(), Count và Clear()
5. Duyệt HashSet và vấn đề thứ tự
6. Chuẩn hóa chuỗi và StringComparer
7. Encapsulation với HashSet
8. Ví dụ hoàn chỉnh EmployeeSkillProfile
9. Các phép toán tập hợp
10. So sánh List, Dictionary và HashSet
11. HashSet với object
12. Lỗi thường gặp
13. Tóm tắt cần nhớ
14. Bài tập StudentCourseRegistration

1. HashSet là gì?

HashSet<T> là collection dùng để lưu một tập hợp các phần tử duy nhất. Nếu thêm một phần tử đã tồn tại, HashSet không thêm lại lần thứ hai.

```
HashSet<string> studentIds = new HashSet<string>();

studentIds.Add("SV01");
studentIds.Add("SV02");
studentIds.Add("SV01");
```

Kết quả vẫn chỉ có SV01 và SV02. HashSet tự loại bỏ sự trùng lặp ở mức collection.

Cú pháp tổng quát

```
HashSet<T> tenTapHop = new HashSet<T>();
```

Trong đó T là kiểu dữ liệu của phần tử được lưu.

```
HashSet<int> numbers = new HashSet<int>();
HashSet<string> skills = new HashSet<string>();
HashSet<string> employeeIds = new HashSet<string>();
```

HashSet khác List ở điểm cốt lõi nào?

Đặc điểm	List<T>	HashSet<T>
Phần tử trùng	Cho phép	Không cho phép
Truy cập theo index	Có	Không
Thứ tự	Phù hợp khi cần thứ tự	Không nên phụ thuộc thứ tự
Kiểm tra tồn tại	Contains()	Contains()

2. Add() trả về bool

Khác với List<T>, method Add() của HashSet trả về một giá trị bool cho biết phần tử có được thêm thành công hay không.

```
HashSet<string> skills = new HashSet<string>();

bool firstResult = skills.Add("C#");
bool secondResult = skills.Add("C#");

Console.WriteLine(firstResult); // True
Console.WriteLine(secondResult); // False
```

Kết quả Add()	Ý nghĩa
true	Phần tử chưa tồn tại và đã được thêm
false	Phần tử đã tồn tại nên không được thêm lại

Cách xử lý kết quả

```
if (skills.Add("C#"))
{
    Console.WriteLine("Skill added successfully.");
}
else
{
    Console.WriteLine("Skill already exists.");
}
```

HashSet không tự ném exception khi phần tử bị trùng. Lần Add() thứ hai chỉ trả về false.

3. Contains(), Remove(), Count và Clear()

Contains() - kiểm tra tồn tại

```
if (skills.Contains("SQL"))
{
    Console.WriteLine("SQL skill exists.");
}
```

Remove() - xóa phần tử

```
bool removed = skills.Remove("Git");

Console.WriteLine(removed
    ? "Skill removed successfully."
    : "Skill not found.");
```

Count và Clear()

```
Console.WriteLine(skills.Count);

skills.Clear();

Console.WriteLine(skills.Count); // 0
```

Contains() và Remove() đều làm việc trực tiếp với giá trị. Không cần tìm object trước như List.

4. Duyệt HashSet và vấn đề thứ tự

Dùng foreach để duyệt HashSet:

```
foreach (string skill in skills)
{
    Console.WriteLine(skill);
}
```

HashSet không hỗ trợ truy cập theo index:

```
skills[0]; // Không hợp lệ
```

Không nên phụ thuộc vào thứ tự phần tử trong HashSet. Nếu bài toán cần thứ tự nhập, vị trí hoặc index, hãy ưu tiên List.

Khi nào HashSet phù hợp?

- Cần bảo đảm dữ liệu không trùng.
- Chỉ cần biết một giá trị có tồn tại hay không.
- Thêm và xóa theo giá trị.
- Không cần truy cập theo vị trí.

Ví dụ ứng dụng

Bài toán	Kiểu HashSet gợi ý
Kỹ năng của nhân viên	HashSet<string>
Mã môn học đã đăng ký	HashSet<string>
Vai trò người dùng	HashSet<string>
Tag sản phẩm	HashSet<string>
Mã sản phẩm yêu thích	HashSet<string>

5. Chuẩn hóa chuỗi và StringComparer

Phân biệt hoa thường mặc định

```
HashSet<string> skills = new HashSet<string>();

skills.Add("C#");
skills.Add("c#");
```

Mặc định, hai chuỗi trên có thể được xem là hai phần tử khác nhau.

Không phân biệt hoa thường

```
HashSet<string> skills = new HashSet<string>(
    StringComparer.OrdinalIgnoreCase);

skills.Add("C#"); // true
skills.Add("c#"); // false
```

Luôn Trim() trước khi thao tác

```
public bool AddSkill(string skill)
{
    if (string.IsNullOrEmpty(skill))
    {
        throw new ArgumentException(
            "Skill cannot be null or whitespace.",
            nameof(skill));
    }

    string normalizedSkill = skill.Trim();
    return _skills.Add(normalizedSkill);
}
```

StringComparer xử lý hoa/thường, còn Trim() xử lý khoảng trắng đầu và cuối. Hai bước này giải quyết hai vấn đề khác nhau.

6. Encapsulation với HashSet

Nguyên tắc đóng gói giống List và Dictionary: không cho bên ngoài sửa collection tùy ý.

Không nên công khai HashSet có thể sửa

```
public HashSet<string> Skills { get; }
```

Vì bên ngoài có thể bỏ qua validation:

```
profile.Skills.Clear();
profile.Skills.Add("");
```

Nên dùng private readonly

```
private readonly HashSet<string> _skills;
```

private ngăn bên ngoài truy cập trực tiếp. readonly ngăn field trở sang một HashSet khác sau khi constructor kết thúc, nhưng nội dung bên trong vẫn có thể Add/Remove.

Cung cấp method có kiểm soát

```
AddSkill()
HasSkill()
RemoveSkill()
DisplaySkills()
```

Ý nghĩa thực sự của encapsulation: mọi thay đổi dữ liệu phải đi qua method của class để class kiểm soát quy tắc hợp lệ.

7. Ví dụ hoàn chỉnh - EmployeeSkillProfile

```
public class EmployeeSkillProfile
{
    public string EmployeeId { get; }
    private readonly HashSet<string> _skills;

    public EmployeeSkillProfile(string employeeId)
    {
        if (string.IsNullOrEmpty(employeeId))
        {
            throw new ArgumentException(
                "Employee Id cannot be null or whitespace.",
                nameof(employeeId));
        }

        EmployeeId = employeeId.Trim();
        _skills = new HashSet<string>(
            StringComparer.OrdinalIgnoreCase);
    }

    public bool AddSkill(string skill)
    {
        ValidateSkill(skill);
        return _skills.Add(skill.Trim());
    }

    public bool HasSkill(string skill)
    {

```

```
ValidateSkill(skill);
return _skills.Contains(skill.Trim());
}

public bool RemoveSkill(string skill)
{
    ValidateSkill(skill);
    return _skills.Remove(skill.Trim());
}

private static void ValidateSkill(string skill)
{
    if (string.IsNullOrEmpty(skill))
    {
        throw new ArgumentException(
            "Skill cannot be null or whitespace.",
            nameof(skill));
    }
}

public void DisplaySkills()
{
    Console.WriteLine($"Skills of {EmployeeId}:");

    if (_skills.Count == 0)
    {
        Console.WriteLine("No skills.");
        return;
    }

    foreach (string skill in _skills)
    {
        Console.WriteLine($"- {skill}");
    }
}
```


Program.cs minh họa

```
EmployeeSkillProfile profile =
new EmployeeSkillProfile("NV01");

bool addedCSharp = profile.AddSkill("C#");
bool addedSql = profile.AddSkill("SQL");
bool addedDuplicate = profile.AddSkill("c#");

Console.WriteLine(addedDuplicate
? "c# added."
: "c# already exists.");

profile.DisplaySkills();

if (profile.HasSkill("sql"))
{
Console.WriteLine("Employee knows SQL.");
}

bool removed = profile.RemoveSkill("SQL");
Console.WriteLine(removed
? "SQL removed successfully."
: "SQL not found.");
```

Do dùng StringComparer.OrdinalIgnoreCase, các giá trị "C#" và "c#" được xem là cùng một kỹ năng.

8. Các phép toán tập hợp

HashSet hỗ trợ các phép toán tương tự tập hợp trong toán học.

```
HashSet<string> backendSkills = new HashSet<string>
{
"C#", "SQL", "ASP.NET Core"
};

HashSet<string> employeeSkills = new HashSet<string>
{
"C#", "Git", "SQL"
};
```

UnionWith() - hợp

Gộp toàn bộ phần tử của hai tập hợp, không trùng.

```
employeeSkills.UnionWith(backendSkills);
```

IntersectWith() - giao

Chỉ giữ các phần tử xuất hiện trong cả hai tập hợp.

```
employeeSkills.IntersectWith(backendSkills);
```

ExceptWith() - hiệu

Loại bỏ các phần tử xuất hiện trong tập hợp còn lại.

```
employeeSkills.ExceptWith(backendSkills);
```

SetEquals() và IsSubsetOf()

```
bool areEqual = employeeSkills.SetEquals(backendSkills);  
bool isSubset = employeeSkills.IsSubsetOf(backendSkills);
```

Các phép toán này làm thay đổi HashSet hiện tại, ngoại trừ các method kiểm tra như SetEquals() và IsSubsetOf().

9. So sánh List, Dictionary và HashSet

Đặc điểm	List<T>	Dictionary<TKey,TValue>	HashSet<T>
Cho phép trùng	Có	Value có thể trùng, Key không	Không
Truy cập	Index	Key	Không có index
Kiểm tra tồn tại	Contains()	ContainsKey()	Contains()
Tìm theo ID	Thường phải duyệt	TryGetValue()	Chỉ biết có/không
Phù hợp khi	Cần danh sách/thứ tự	Tra cứu theo khóa	Loại bỏ trùng

Cách chọn collection

Cần thứ tự hoặc index -> List<T>
 Cần tra cứu object theo khóa -> Dictionary<TKey,TValue>
 Cần tập hợp giá trị duy nhất -> HashSet<T>

10. Lưu ý khi HashSet chứa object

```
HashSet<Employee> employees = new HashSet<Employee>();

Employee e1 = new Employee("NV01", "Hung");
Employee e2 = new Employee("NV01", "Hung");

employees.Add(e1);
employees.Add(e2);
```

Dù dữ liệu giống nhau, HashSet có thể vẫn xem đây là hai object khác nhau vì chúng là hai reference khác nhau.

Để xác định trùng theo Id, cần học thêm:

- Equals()
- GetHashCode()
- IEquatable
- IEqualityComparer

Ở giai đoạn hiện tại, khi chỉ cần kiểm tra mã không trùng, nên dùng HashSet<string> thay vì HashSet<Employee>.

11. Các lỗi thường gặp

Lỗi 1 - Mong Add() ném exception khi trùng

```
_skills.Add("C#");
_skills.Add("C#");
```

Lần thứ hai chỉ trả về false.

Lỗi 2 - Cố truy cập bằng index

```
_skills[0]; // Sai
```

Lỗi 3 - Không Trim()

```
_skills.Add("C#");
_skills.Add(" C# ");
```

Lỗi 4 - Không xử lý hoa thường

```
new HashSet<string>(StringComparer.OrdinalIgnoreCase);
```

Lỗi 5 - Để HashSet là public

```
public HashSet<string> Skills { get; }
```

Lỗi 6 - Dùng HashSet khi cần thứ tự

Nếu đầu ra phải giữ đúng thứ tự nhập hoặc cần index, dùng List sẽ phù hợp hơn.

12. Tóm tắt cần nhớ

Member	Ý nghĩa
Add(value)	Thêm phần tử; trả về true/false
Contains(value)	Kiểm tra tồn tại
Remove(value)	Xóa phần tử; trả về true/false
Count	Số phần tử
Clear()	Xóa toàn bộ
UnionWith()	Phép hợp
IntersectWith()	Phép giao
ExceptWith()	Phép hiệu
SetEquals()	Kiểm tra hai tập hợp bằng nhau

Quy tắc cốt lõi:

1. HashSet lưu phần tử không trùng.
2. Add() trả về bool.
3. Không có index và không nên phụ thuộc thứ tự.
4. Luôn validation và Trim() chuỗi.
5. Dùng StringComparer.OrdinalIgnoreCase khi không muốn phân biệt hoa thường.

6. Giữ collection ở dạng private readonly.

13. Bài tập 8.3 - Quản lý môn học đã đăng ký

Tạo class `StudentCourseRegistration` để quản lý các mã môn học mà một sinh viên đã đăng ký. Một mã môn học không được xuất hiện hai lần.

Thuộc tính và collection

```
public string StudentId { get; }  
  
private readonly HashSet<string> _courseCodes;
```

Khởi tạo `HashSet` không phân biệt hoa thường:

```
_courseCodes = new HashSet<string>(  
    StringComparer.OrdinalIgnoreCase);
```

Method 1 - RegisterCourse

```
public bool RegisterCourse(string courseCode)
```

- `courseCode` không được null, rỗng hoặc chỉ có khoảng trắng.
- Dùng `Trim()`.
- Trả về trực tiếp kết quả của `Add()`.

Method 2 - IsCourseRegistered

```
public bool IsCourseRegistered(string courseCode)
```

- Validation `courseCode`.
- Dùng `Contains()`.

Method 3 - UnregisterCourse

```
public bool UnregisterCourse(string courseCode)
```

- Validation `courseCode`.
- Dùng `Remove()`.

Method 4 - DisplayRegisteredCourses

```
public void DisplayRegisteredCourses()
```

- Hiển thị `StudentId`.
- Kiểm tra `HashSet` rỗng.
- Duyệt bằng `foreach`.

14. Yêu cầu trong Program.cs

Sau khi hoàn thành class, thực hiện lần lượt các yêu cầu sau:

- Tạo một StudentCourseRegistration.
- Đăng ký ít nhất ba mã môn học.
- Hiển thị toàn bộ môn học đã đăng ký.
- Thử đăng ký lại một mã đã tồn tại.
- Kiểm tra và thông báo kết quả trả về của Add().
- Kiểm tra một môn học bằng Contains().
- Hủy đăng ký một môn học.
- Kiểm tra kết quả xóa.
- Thử xóa một môn học không tồn tại.
- Hiển thị lại tập hợp sau khi xóa.

Bắt buộc sử dụng

```
HashSet<string>
Add()
Contains()
Remove()
Count
foreach
StringComparer.OrdinalIgnoreCase
```

Chưa dùng LINQ. Chưa cần UnionWith() hoặc IntersectWith() trong bài tập đầu tiên.

Checklist trước khi nộp bài

Kiểm tra	Đã làm
StudentId và courseCode được validation	☐
Chuỗi được Trim()	☐
HashSet dùng OrdinalIgnoreCase	☐
Đăng ký trùng trả về false	☐
Contains() kiểm tra đúng	☐
Remove() trả về đúng bool	☐
Không truy cập bằng index	☐
Collection là private readonly	☐
Program kiểm tra cả trường hợp thành công và thất bại	☐

Hết Phần 8.3 - HashSet